TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP VIỆT – HUNG
KHOA:CÔNG NGHỆ THÔNG TIN



NHÓM 1

XÂY DỰNG CẤU TRÚC MÔ TẢ HOẠT ĐỘNG CỦA HÀNG ĐỢI (QUEUE)

ĐỒ ÁN HỌC PHẦN I
CHUYÊN NGÀNH: CÔNG NGHỆ THÔNG TIN

Giáo viên hướng dẫn : Chu Thị Thanh Xuân
Họ tên sinh viên : Mai Tiến Thắng 2201153
: Tẩn Láo Tả 2200689
: Ma Hoàng Việt 2200570
: Sùng A Dũng 2200694
: Lù A Hạnh 2201020
Lớp : K46B – CNTT

HÀ NỘI, NĂM 2024

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP VIỆT – HUNG
KHOA:CÔNG NGHỆ THÔNG TIN

XÂY DỰNG CẤU TRÚC MÔ TẢ HOẠT ĐỘNG CỦA HÀNG ĐỢI (QUEUE)

ĐỒ ÁN HỌC PHẦN I
CHUYÊN NGÀNH: CÔNG NGHỆ THÔNG TIN

Giáo viên hướng dẫn : Chu Thị Thanh Xuân
Nhóm thực hiện : Nhóm 1
Họ tên sinh viên : Mai Tiến Thắng 2201153
: Tẩn Láo Tả 2200689
: Ma Hoàng Việt 2200570
: Sùng A Dũng 2200694
: Lù A Hạnh 2201020
Lớp : K46B – CNTT

HÀ NỘI, NĂM 2024

PHIẾU GIAO ĐỀ TÀI

Đề tài số: 8249587

Tên đề tài: Xây dựng cấu trúc mô tả hoạt động của hàng đợi (Queue)

Yêu cầu:

1.Yêu cầu về kiến thức, kỹ năng:

- Hiểu được cấu trúc, nguyên lý hoạt động của hàng đợi.
- Biết sử dụng ngôn ngữ C, C++

2. Yêu cầu về chương trình:

1/ Cài đặt cấu trúc hàng đợi bằng mảng và bằng danh sách liên kết.

2/ Viết hàm PrintList(): Nhập dãy số nguyên từ bàn phím và đưa vào hàng đợi nếu gặp số -1 thì in ra màn hình danh sách các phần tử lấy ra khỏi hàng đợi.

3/ Viết hàm tìm phần tử lớn nhất, nhỏ nhất lấy ra khỏi hàng đợi và in ra màn hình.

4/ Viết hàm tính tổng các phần tử lấy ra khỏi hàng đợi.

5/ Tìm phần tử có giá trị x (nhập từ bàn phím) có trong hàng đợi không và đưa ra thông báo.

6/ Viết hàm thực hiện yêu cầu: Cho 1 chuỗi mô tả các thao tác trên queue gồm các ký tự chữ cái và dấu * với quy tắc như sau:

- Chữ cái tượng trưng cho thao tác thêm chữ cái tương ứng vào queue
- Dấu * tượng trưng cho thao tác lấy nội dung một phần tử ra khỏi queue

Thực hiện chuỗi thao tác này trên queue theo thứ tự từ trái sang phải.Sau khi hoàn tất, tiến hành lấy lần lượt tất cả các phần tử còn lại trong queue, hãy cho biết nội dung lấy được.

Ví dụ: AAA**BCC*D

Kết quả khi thao tác trên Queue là : DCBA

7/ Nâng cao: Cài đặt cấu trúc sinh viên gồm Họ tên, Ngày sinh, Giới tính. Lần lượt đưa sinh viên vào hàng đợi.

3. Nộp kết quả:

- Quyển báo cáo (theo mẫu của bộ môn)
- Mã nguồn chương trình, file báo cáo.

4. Đánh giá:

- Mức đạt, Khá: Hoàn thành các yêu cầu 1,2,3,4,5,6
- Mức khá, tốt: Hoàn thành yêu cầu 7.

5. Các yêu cầu khác:

- Gửi lịch làm việc cho GVHD
- Mỗi tuần phải liên hệ với GV hướng dẫn ít nhất 1 lần qua email.

Mẫu DA03

TRƯỜNG ĐHCN VIỆT – HUNG

KHOA CÔNG NGHỆ THÔNG TIN

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập - Tự do – Hạnh phúc

NHẬN XÉT ĐỒ ÁN HỌC PHẦN…..

Chuyên ngành : …….

(Nhận xét của GV hướng dẫn ☐ Nhận xét của GV phản biện ☐)

Họ tên sinh viên: .....

Tên đề tài: .....

.....

Người nhận xét (họ tên, học hàm, học vị): .....

Đơn vị công tác (nếu có): .....

.....

Ý KIẾN NHẬN XÉT

1. Về nội dung & đánh giá thực hiện nhiệm vụ nghiên cứu của đề tài:

.....

.....

.....

.....

2. Về phương pháp nghiên cứu, độ tin cậy của các số liệu:

.....

.....

.....

.....

3. Về kết quả của đề tài:

.....

.....

.....

.....

.....

4. Những thiếu sát và vấn đề cần làm rõ (nếu có): .....

.....

.....

.....

.....

.....

5. Ý kiến kết luận (mức độ đáp ứng yêu cầu đối với ĐA/KL tốt nghiệp)

.....

.....

.....

.....

6. Câu hỏi người nhận xét dành cho học viên (nếu có):

.....

.....

.....

.....

✍ Đánh giá điểm:

Kết luận: ☐ Đồng ý / ☐ Không đồng ý cho phép sinh viên được tham dự bảo vệ kết quả trước hội đồng.

Ngày……tháng……năm 2024

NGƯỜI NHẬN XÉT

(chữ ký & họ tên)

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến tất cả Quý Thầy/Cô trong khoa Công nghệ Thông tin, trường Đại học Công Nghiệp Việt - Hung, đã tận tình giảng dạy và truyền đạt những kiến thức quý báu trong suốt thời gian học tập tại trường.

Đặc biệt, chúng em xin gửi lời tri ân đến Cô Chu Thị Thanh Xuân, người đã hướng dẫn và hỗ trợ chúng em rất nhiều trong quá trình thực hiện đề tài "Xây dựng cấu trúc mô tả hàng đợi Queue". Những góp ý, lời khuyên và sự tận tâm của Cô đã giúp chúng tôi hoàn thiện đề tài này một cách tốt nhất.

Công nghệ thông tin là một trong những ngành khoa học ngày càng được quan tâm và sử dụng rộng rãi trong mọi lĩnh vực của cuộc sống. Với những ưu điểm mạnh có thể ứng dụng được nhiều trong cuộc sống thì công nghệ thông tin đã giúp cho công việc quản lý được dễ dàng hơn. Để quản lý một công việc cụ thể của một cơ quan nào đó thì người quản lý cần thực hiện rất nhiều động tác. Nhờ có các phần mềm thì công việc đó trở nên đơn giản hơn rất nhiều.

Chúng em cũng xin cảm ơn các bạn trong nhóm nghiên cứu và các bạn sinh viên trong lớp đã luôn sẵn sàng trao đổi, thảo luận và chia sẻ những ý tưởng, góp phần quan trọng vào sự thành công của đề tài.

Cuối cùng, chúng em xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn bên cạnh, động viên và ủng hộ tôi trong suốt quá trình học tập và nghiên cứu.

Chúng em xin chân thành cảm ơn !

NHẬT KÝ THỰC HIỆN ĐỀ TÀI

Thời gian	Công việc	Ghi chú
06/04/2024	Nhận đề tài	
07/04/2024	Phân công nhiệm vụ tìm hiểu về đề tài	
02/05/2024	Nghiên cứu, tìm hiểu về đề tài	1. Tìm hiểu về mảng và danh sách tuyến tính: Mai Tiến Thắng Ma Hoàng Việt 2. Tìm hiểu về danh sách liên kết đơn, danh sách liên kết vòng: Sùng A Dũng, Tẩn Láo Tả, Lù A Hạnh
06/05/2024	Tìm hiểu cấu trúc hàng đợi: . Định nghĩa, các phép toán trên hàng đợi	
13/05/2024	Tìm hiểu và cài đặt hàng đợi bằng mảng (Khởi tạo, thêm, xóa, kiểm tra rỗng, đầy)	
18/05/2024	Tìm hiểu và cài đặt hàng đợi bằng danh sách liên kết (Khởi tạo, thêm, xóa, kiểm tra rỗng, đầy)	
20/05/2024	Ứng dụng cấu trúc hàng đợi để nhập dãy các số nguyên từ bàn phím và đưa vào hàng đợi, nếu gặp số -1 thì in ra màn hình danh sách các phần tử lấy ra khỏi hàng đợi.	
21/05/2024	Viết hàm tìm phần tử lớn nhất lấy ra khỏi hàng đợi	
24/05/2024	Viết hàm tính tổng các phần tử lấy ra khỏi hàng đợi	
26/05/2024	Viết hàm tìm phần tử có giá trị x lấy ra khỏi hàng đợi	
27/05/2024	Thực hiện yêu cầu trong phiếu giao đề tài	
28/05/2024	Cài đặt danh sách sinh viên sử dụng hàng đợi (Nâng cao)	
04/06/2024	Hoàn thiện báo cáo theo mẫu bộ môn	

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI 3
NHẬN XÉT ĐỒ ÁN HỌC PHẦN….. 4
LỜI CẢM ƠN 6
NHẬT KÝ THỰC HIỆN ĐỀ TÀI 7
MỤC LỤC 8
DANH MỤC CHỮ VIẾT TẮT 10
DANH MỤC BẢNG BIỂU 11
DANH MỤC HÌNH VẼ 12
MỞ ĐẦU 13
CHƯƠNG I: KHÁI QUÁT VỀ MẢNG, DANH SÁCH LIÊN KẾT VÀ TỔNG QUAN VỀ HÀNG ĐỢI (QUEUE) 15
1.1. Khái quát về Mảng và Danh sách tuyến tính 15
1.1.1. Mảng (Array) 15
1.1.2. Danh sách tuyến tính (Linked List) 16
1.1.3. Tìm hiểu về danh sách liên kết đơn (Singly Linked List) 16
1.1.4. Danh sách liên kết vòng (Circular Linked List) 17
1.2. Khái niệm và các phép toán trên Hàng đợi 18
1.2.1. Khái niệm Hàng đợi Queue 18
1.2.2. Các phép toán trên Hàng đợi 19
CHƯƠNG II: THIẾT KẾ VÀ TRIỂN KHAI HÀNG ĐỢI 20
2.1. Cài đặt hàng đợi bằng mảng (Khởi tạo, thêm, xóa, kiểm tra rỗng, đầy) 20
2.1.1. Khởi tạo hàng đợi (Initialization) 20
2.1.2. Thêm phần tử vào hàng đợi (enqueue) 20
2.1.3. Xóa phần tử khỏi hàng đợi (dequeue) 21
2.1.4. Kiểm tra hàng đợi rỗng (isEmpty) 21
2.1.5. Kiểm tra hàng đợi đầy (isFull) 21
2.2. Cài đặt hàng đợi bằng danh sách liên kết (Khởi tạo, thêm, xóa, kiểm tra rỗng, đầy) 21
2.2.1. Khởi tạo hàng đợi 21
2.2.2. Thêm phần tử vào hàng đợi (enqueue) 22
2.2.3. Xóa phần tử khỏi hàng đợi (dequeue) 22
2.2.4. Kiểm tra hàng đợi rỗng 22
2.2.5. Kiểm tra hàng đợi đầy 23
2.3. Ứng dụng hàng đợi trong c++ 23
2.3.1. Hàm main 23
CHƯƠNG III: ỨNG DỤNG VÀ ĐÁNH GIÁ HIỆU QUẢ CỦA HÀNG ĐỢI 24
3.1. Các bài toán ứng dụng hàng đợi 24
3.1.1. Ứng dụng cấu trúc hàng đợi để nhập dãy các số nguyên từ bàn phím và đưa vào hàng đợi, nếu gặp số -1 thì in ra màn hình danh sách các phần tử lấy ra khỏi hàng đợi 24
3.1.2. Viết hàm tìm phần tử lớn nhất lấy ra khỏi hàng đợi 25
3.1.3. Viết hàm tính tổng các phần tử lấy ra khỏi hàng đợi 25
3.1.4. Viết hàm tìm phần tử có giá trị x lấy ra khỏi hàng đợi 26
3.1.5. Cho 1 chuỗi mô tả các thao tác trên queue gồm các ký tự chữ cái và dấu * 26
3.2. Cài đặt danh sách sinh viên sử dụng hàng đợi (Nâng cao) 27

3.3. Đánh giá và hướng phát triển trong tương lai 31
3.3.1. Đánh giá 31
3.3.2. Hướng phát triển trong tương lai 31
KẾT LUẬN 32
1. Các kết quả đạt được 32
2. Ưu và nhược điểm của đề tài 32
TÀI LIỆU THAM KHẢO 33
PHỤ LỤC 34
1. Code chạy chương trình hàng đợi (Queue) 34
2. Code chạy chương trình cài đặt cấu trúc sinh viên gồm Họ tên, Ngày sinh, Giới tính, lần lượt đưa sinh viên vào hàng đợi. 37

DANH MỤC CHỮ VIẾT TẮT

TT	Tên viết tắt	Tên đầy đủ	Dịch ra Tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	ML	Machine Learning	Học Máy
3	O(n)	Linear Time	Thời gian thực hiện tăng tuyến tính với kích thước của đầu vào
4	FIFO	First In, First Out	Vào trước, ra trước
5	C++	C with Classes	Ngôn ngữ lập trình cấp cao
6	IoT	Internet of Things	Mạng lưới các thiết bị vật lý được kết nối với internet

DANH MỤC BẢNG BIỂU

DANH MỤC HÌNH VẼ

Hình 1.1. Mô tả cấu trúc hàng đợi(Queue)
Hình 1.2. Biểu diễn hàng đợi dưới dạng cấu trúc dữ liệu
Hình 2.1. Mô tả khởi tạo hàng đợi
Hình 2.2. Mô tả thêm phần tử vào hàng đợi
Hình 2.3. Mô tả xóa phần tử khỏi hàng đợi
Hình 2.4. Mô tả hàng đợi rỗng
Hình 2.5. Khởi tạo hàng đợi bằng danh sách liên kết
Hình 2.6. Mô tả thêm phần tử vào hàng đợi bằng danh sách liên kết
Hình 2.7. Mô tả xóa phần tử khỏi hàng đợi bằng danh sách liên kết
Hình 2.8. Mô tả hằng đợi rỗng danh sách liên kết
Hình 2.9. Mô tả hằng đợi đầy danh sách liên kết
Hình 2.10. Mô tả hàm main trong C++
Hình 2.11. Mô tả hàm hiển thị các phần tử hàng đợi trong c++
Hình 3.1. Mô tả hàm gặp -1 thì in ra màn hình danh sách hàng đợi
Hình 3.2. Mô tả hàm tìm giá trị lớn nhất trong hàng đợi bằng c++
Hình 3.3. Mô tả hàm tính tổng các phần tử ra khỏi hàng đợi
Hình 3.4. Mô tả hàm lấy giá trị x ra khỏi hàng đợi
*Hình 3.5. Mô tả thao tác trên queue gồm các ký tự chữ cái và dấu **
Hình 3.6. Mô tả khởi tạo lưu trữ thông tin cơ bản sinh viên
Hình 3.7. Mô tả lớp queue quản lý hàng đợi các đối tượng Sinhvien
Hình 3.8. Mô tả kiểm tra rỗng, đầy trong danh sách sinh viên
Hình 3.9. Mô tả hàm thêm trong danh sách sinh viên
Hình 3.10. Mô tả hàm xóa trong danh sách sinh viên
Hình 3.11. Mô tả hàm hiển thị danh sách ra màn hình
Hình 3.12. Mô tả tìm và xóa sinh viên theo mã sinh viên
Hình 3.13. Mô tả tìm và xóa sinh viên theo tên sinh viên
Hình 3.14. Mô tả hàm main trong cấu trúc danh sách sinh viên sử dụng hàng đợi

MỞ ĐẦU

I. Đặt vấn đề

- Trong các hệ thống xử lý công việc, việc quản lý và ưu tiên các tác vụ một cách hiệu quả là rất quan trọng để đảm bảo hệ thống hoạt động mượt mà và đáp ứng yêu cầu thời gian thực. Hàng đợi ưu tiên là một công cụ quan trọng trong việc này, nhưng việc tối ưu hóa hàng đợi ưu tiên để đạt được hiệu suất cao nhất vẫn là một thách thức.

II. Mục tiêu nghiên cứu của đề tài

- Tìm hiểu về cấu trúc dữ liệu hàng đợi, các loại hàng đợi và các ứng dụng của chúng trong lĩnh vực khoa học máy tính.
- Hàng đợi là một cấu trúc dữ liệu quan trọng, được sử dụng rộng rãi trong nhiều thuật toán và hệ thống máy tính.
- Nghiên cứu cấu trúc và hoạt động của hàng đợi ưu tiên trong hệ thống xử lý công việc.
- Đánh giá hiệu suất của các thuật toán hiện tại và xác định các điểm yếu.
- Đề xuất các thuật toán mới hoặc cải tiến các thuật toán hiện có để tối ưu hóa hàng đợi ưu tiên.
- Thực nghiệm và so sánh hiệu suất của các thuật toán được đề xuất với các thuật toán hiện tại.

III. Nhiệm vụ của đề tài

- Tìm hiểu lý thuyết: Đọc các tài liệu, sách giáo trình và bài báo khoa học về hàng đợi để hiểu rõ khái niệm và nguyên lý hoạt động.
- Phân loại hàng đợi: Tổng hợp và phân tích các loại hàng đợi (đơn giản, ưu tiên, vòng, hai đầu).
- Tìm hiểu ứng dụng thực tế: Nghiên cứu cách hàng đợi được sử dụng trong hệ thống xử lý công việc, quản lý tài nguyên mạng, thuật toán tìm kiếm đồ thị, và xử lý dữ liệu phân tán.
- Viết bài phân tích: Đưa ra các ví dụ cụ thể và mô tả cách hàng đợi cải thiện hiệu suất và tối ưu hóa các hệ thống này.
- Nghiên cứu thuật toán tối ưu: Tìm hiểu các thuật toán và cấu trúc hàng đợi mới nhằm cải thiện hiệu suất.
- Ứng dụng trong AI và ML: Khám phá các ứng dụng tiềm năng của hàng đợi trong trí tuệ nhân tạo và học máy.
- Tổng hợp kết quả nghiên cứu: Viết báo cáo chi tiết về các phát hiện, phân tích và so sánh.

IV. Kết quả đạt được của đề tài

- Hiểu biết sâu rộng: Đạt được kiến thức chuyên sâu về khái niệm, nguyên lý hoạt động và phân loại hàng đợi.

- Cài đặt và thao tác: Hiểu rõ cách cài đặt và thực hiện các thao tác cơ bản (enqueue, dequeue, peek, isEmpty) trên hàng đợi bằng cả mảng và danh sách liên kết.

- Hiệu suất thực tiễn: Đánh giá hiệu suất của các cài đặt hàng đợi qua các thí nghiệm thực tiễn, chỉ ra khi nào và tại sao một cài đặt lại hiệu quả hơn cái còn lại.

- Phân tích ứng dụng: Đánh giá các ứng dụng thực tế của hàng đợi trong hệ thống xử lý công việc, quản lý tài nguyên mạng, thuật toán tìm kiếm đồ thị và xử lý dữ liệu phân tán.

- Thuật toán tối ưu: Tìm hiểu và đề xuất các thuật toán và cấu trúc hàng đợi mới để tối ưu hóa hiệu suất.

- Kết quả đạt được của đề tài này không chỉ là sự hiểu biết sâu sắc về hàng đợi mà còn bao gồm khả năng ứng dụng kiến thức vào thực tiễn, khả năng phân tích và so sánh các cấu trúc dữ liệu khác nhau, cũng như khả năng trình bày và truyền đạt kết quả nghiên cứu một cách rõ ràng và thuyết phục.

V. Bố cục của đề tài

Đề tài gồm 3 chương như sau:

Chương I : Khái quát về Mảng, Danh sách liên kết và Tổng quan về Hàng đợi (Queue).

Chương II : Thiết kế và Triển khai Hàng đợi.

Chương III : Ứng dụng và Đánh giá hiệu quả của Hàng đợi.

CHƯƠNG I: KHÁI QUÁT VỀ MẢNG, DANH SÁCH LIÊN KẾT VÀ TỔNG QUAN VỀ HÀNG ĐỢI (QUEUE)

Tóm tắt chương 1

Mảng là một cấu trúc dữ liệu lưu trữ các phần tử liên tiếp trong bộ nhớ, cho phép truy cập nhanh chóng bằng chỉ số, nhưng có kích thước cố định và khó chèn/xóa phần tử. Danh sách liên kết gồm các nút chứa dữ liệu và liên kết tới nút kế tiếp, cho phép kích thước linh hoạt và dễ chèn/xóa, nhưng truy cập chậm hơn do phải duyệt qua từng nút. Hàng đợi (Queue) là một cấu trúc dữ liệu theo nguyên tắc "vào trước ra trước" (FIFO), thường được dùng để quản lý các tác vụ theo thứ tự. Hàng đợi có thể được triển khai bằng mảng hoặc danh sách liên kết, mỗi cách đều có ưu và nhược điểm riêng về quản lý bộ nhớ và hiệu suất.

1.1. Khái quát về Mảng và Danh sách tuyến tính

1.1.1. Mảng (Array)

Định nghĩa:

Mảng là một cấu trúc dữ liệu bao gồm một tập hợp các phần tử có cùng kiểu dữ liệu, được lưu trữ trong các ô nhớ liên tiếp nhau. Các phần tử trong mảng có thể được truy cập thông qua chỉ số của chúng.

Đặc điểm:

- Kích thước cố định: Khi mảng được khởi tạo, kích thước của nó không thể thay đổi.
- Truy cập ngẫu nhiên: Có thể truy cập bất kỳ phần tử nào trong mảng một cách trực tiếp thông qua chỉ số, với thời gian truy cập trung bình là O(1).
- Lưu trữ liên tiếp: Các phần tử của mảng được lưu trữ liên tiếp nhau trong bộ nhớ, giúp tối ưu hóa việc truy cập và sử dụng bộ nhớ.

Ưu điểm:

- Truy cập phần tử nhanh chóng.
- Đơn giản trong việc quản lý và thao tác.

Nhược điểm:

- Kích thước cố định, không thể thay đổi sau khi khởi tạo.
- Phải biết trước số lượng phần tử cần lưu trữ.
- Việc chèn hoặc xóa phần tử phức tạp và tốn thời gian, đặc biệt là đối với các mảng lớn.

1.1.2. Danh sách tuyến tính (Linked List)

Định nghĩa:

Danh sách tuyến tính (hay danh sách liên kết) là một cấu trúc dữ liệu gồm một chuỗi các phần tử, trong đó mỗi phần tử chứa một giá trị và một con trỏ trỏ đến phần tử kế tiếp.

Đặc điểm:

- Kích thước động: Danh sách tuyến tính có thể thay đổi kích thước linh hoạt, thêm hoặc bớt phần tử một cách dễ dàng.

- Truy cập tuần tự: Để truy cập một phần tử bất kỳ, cần phải duyệt qua các phần tử trước đó, với thời gian truy cập trung bình là O(n).

- Bộ nhớ phân tán: Các phần tử không cần phải được lưu trữ liên tiếp trong bộ nhớ.

Ưu điểm:

- Dễ dàng thay đổi kích thước.

- Việc chèn và xóa phần tử dễ dàng, đặc biệt là khi thực hiện ở đầu danh sách.

Nhược điểm:

- Truy cập phần tử chậm hơn so với mảng.

- Sử dụng thêm bộ nhớ cho các con trỏ.

- Dễ gặp lỗi nếu không quản lý tốt các con trỏ.

1.1.3. Tìm hiểu về danh sách liên kết đơn (Singly Linked List)

Khái niệm

Danh sách liên kết đơn (Singly Linked List) là một cấu trúc dữ liệu bao gồm một chuỗi các phần tử (node). Mỗi phần tử trong danh sách chứa một giá trị và một con trỏ (pointer) trỏ đến phần tử kế tiếp trong danh sách. Phần tử cuối cùng trong danh sách liên kết đơn có con trỏ trỏ đến null, chỉ ra rằng đó là phần tử cuối cùng.

Cấu trúc của một Node

Mỗi node trong danh sách liên kết đơn bao gồm hai thành phần chính:

- Dữ liệu (data): Chứa giá trị của node.

- Con trỏ (next): Trỏ đến node kế tiếp trong danh sách.

Ưu điểm và Nhược điểm

Ưu điểm:

- Kích thước động: Dễ dàng thay đổi kích thước danh sách bằng cách thêm hoặc xóa các node.

- Chèn và xóa nhanh chóng: Các thao tác chèn/xóa node đầu tiên hoặc ở giữa danh sách có thể thực hiện một cách nhanh chóng và hiệu quả hơn so với mảng.

Nhược điểm:

- Truy cập phần tử chậm: Để truy cập một phần tử cụ thể, phải duyệt qua từng node từ đầu danh sách, với thời gian trung bình O(n).

- Sử dụng thêm bộ nhớ: Mỗi node cần thêm bộ nhớ cho con trỏ next.

- Không có truy cập ngẫu nhiên: Không thể truy cập trực tiếp vào một phần tử bất kỳ như trong mảng.

Ứng dụng của Danh sách Liên kết Đơn

Danh sách liên kết đơn được sử dụng trong nhiều ứng dụng lập trình và giải thuật, chẳng hạn như:

- Triển khai các cấu trúc dữ liệu phức tạp hơn như hàng đợi (queue), ngăn xếp (stack), và danh sách liên kết kép (doubly linked list).

- Quản lý bộ nhớ động và thực hiện các thao tác chèn/xóa nhanh chóng trong các hệ thống yêu cầu thay đổi dữ liệu liên tục.

- Làm nền tảng cho các giải thuật liên quan đến chuỗi (string) và đồ thị (graph).

1.1.4. Danh sách liên kết vòng (Circular Linked List)

Khái niệm

Danh sách liên kết vòng (Circular Linked List) là một biến thể của danh sách liên kết, trong đó node cuối cùng trỏ lại node đầu tiên, tạo thành một vòng khép kín. Danh sách liên kết vòng có thể là danh sách liên kết đơn hoặc danh sách liên kết kép.

Cấu trúc của Danh sách Liên kết Vòng

Mỗi node trong danh sách liên kết vòng bao gồm hai thành phần chính:

- Dữ liệu (data): Chứa giá trị của node.

- Con trỏ (next): Trỏ đến node kế tiếp trong danh sách. Đối với node cuối cùng, con trỏ next trỏ đến node đầu tiên trong danh sách.

Ưu điểm và Nhược điểm

Ưu điểm:

- Duyệt tuần hoàn: Danh sách liên kết vòng cho phép duyệt tuần hoàn mà không cần kiểm tra điều kiện đặc biệt để quay lại đầu danh sách.

- Kích thước động: Dễ dàng thay đổi kích thước danh sách bằng cách thêm hoặc xóa các node.

- Chèn và xóa nhanh chóng: Các thao tác chèn/xóa node đầu tiên hoặc cuối cùng có thể thực hiện một cách nhanh chóng và hiệu quả hơn so với mảng.

Nhược điểm:

- Truy cập phần tử chậm: Để truy cập một phần tử cụ thể, phải duyệt qua từng node từ đầu danh sách, với thời gian trung bình O(n).

- Sử dụng thêm bộ nhớ: Mỗi node cần thêm bộ nhớ cho con trỏ next.

- Không có truy cập ngẫu nhiên: Không thể truy cập trực tiếp vào một phần tử bất kỳ như trong mảng.

Ứng dụng của Danh sách Liên kết Vòng

Danh sách liên kết vòng được sử dụng trong nhiều ứng dụng lập trình và giải thuật, chẳng hạn như:

- Các hệ thống yêu cầu duyệt tuần hoàn: Các ứng dụng như các hệ thống quản lý tài nguyên (round-robin scheduling) và các trò chơi (game) sử dụng danh sách liên kết vòng để luân phiên các tài nguyên hoặc người chơi.

- Triển khai các cấu trúc dữ liệu phức tạp hơn: Như hàng đợi vòng (circular queue) trong các hệ thống nhúng và các bộ đệm vòng (circular buffer) trong truyền thông dữ liệu.

- Quản lý các công việc định kỳ: Các hệ thống cần quản lý và thực thi các công việc định kỳ có thể sử dụng danh sách liên kết vòng để dễ dàng luân phiên giữa các công việc.

1.2. Khái niệm và các phép toán trên Hàng đợi

1.2.1. Khái niệm Hàng đợi Queue

Hàng đợi (Queue) là một cấu trúc dữ liệu trừu tượng hoạt động theo nguyên tắc "vào trước, ra trước" (First In, First Out - FIFO). Điều này có nghĩa là phần tử được thêm vào đầu tiên sẽ là phần tử được lấy ra đầu tiên. Hàng đợi giống như một hàng chờ trong đời sống hàng ngày, nơi mà người đến trước sẽ được phục vụ trước.



Hình 1.1. Mô tả cấu trúc hàng đợi(Queue)

Cấu trúc của Hàng đợi

- Front (Đầu hàng đợi): Chỉ vị trí của phần tử đầu tiên trong hàng đợi.

- Rear (Cuối hàng đợi): Chỉ vị trí của phần tử cuối cùng trong hàng đợi.

- Queue (Mảng hoặc Danh sách chứa các phần tử): Lưu trữ các phần tử của hàng đợi.



Hình 1.2. Biểu diễn hàng đợi dưới dạng cấu trúc dữ liệu

1.2.2. Các phép toán trên Hàng đợi

- Enqueue (Thêm phần tử): Thêm một phần tử mới vào cuối hàng đợi.
- Dequeue (Xóa phần tử): Loại bỏ phần tử ở đầu hàng đợi.
- isEmpty (Kiểm tra rỗng): Kiểm tra xem hàng đợi có rỗng không.
- isFull (Kiểm tra đầy): Kiểm tra xem hàng đợi có đầy không.
- Peek (Xem phần tử đầu): Xem giá trị của phần tử ở đầu hàng đợi mà không loại bỏ nó.

CHƯƠNG II: THIẾT KẾ VÀ TRIỂN KHAI HÀNG ĐỢI

Tóm tắt chương 2

Hàng đợi (Queue) là cấu trúc dữ liệu theo nguyên tắc FIFO (First In, First Out). Có hai phương pháp chính để triển khai hàng đợi: sử dụng mảng và danh sách liên kết. Khi dùng mảng, hàng đợi có kích thước cố định và có thể gây lãng phí bộ nhớ nếu không được sử dụng hết. Với danh sách liên kết, hàng đợi có kích thước động, cho phép chèn và xóa phần tử linh hoạt hơn. Các thao tác cơ bản của hàng đợi gồm: enqueue (thêm phần tử), dequeue (xóa phần tử), isEmpty (kiểm tra hàng đợi trống), isFull (kiểm tra hàng đợi đầy, nếu dùng mảng), và peek (xem phần tử đầu tiên). Việc chọn cách triển khai phụ thuộc vào yêu cầu cụ thể về hiệu suất và quản lý bộ nhớ trong ứng dụng thực tế.

2.1. Cài đặt hàng đợi bằng mảng (Khởi tạo, thêm, xóa, kiểm tra rỗng, đầy)

2.1.1. Khởi tạo hàng đợi (Initialization)

- Khởi tạo một mảng queue với kích thước cố định maxSize.
- Đặt con trỏ front và rear ban đầu là -1 để chỉ ra rằng hàng đợi đang rỗng.
- Destructor ~Queue() được sử dụng để giải phóng bộ nhớ khi hàng đợi không còn được sử dụng.

```
1  Queue(int size) {
2      maxSize = size;
3      queue = new int[maxSize];
4      front = -1;
5      rear = -1;
6  }
7
8  ~Queue() {
9      delete[] queue;
10 }
```

Hình 2.1. Mô tả khởi tạo hàng đợi

2.1.2. Thêm phần tử vào hàng đợi (enqueue)

- Nếu hàng đợi đầy, in thông báo và không thêm phần tử.
- Nếu hàng đợi rỗng, đặt front về 0.
- Tăng giá trị rear và thêm phần tử vào vị trí rear.

```
31 void enqueue(int element) {
32     if (isFull()) {
33         cout << "Hang doi day !!!" << endl;
34         return;
35     }
36     if (isEmpty()) {
37         front = 0;
38     }
39     rear++;
40     queue[rear] = element;
41     cout << "Da them phan tu : " << element << endl;
42 }
```

Hình 2.2. Mô tả thêm phần tử vào hàng đợi

2.1.3. Xóa phần tử khỏi hàng đợi (dequeue)

- Nếu hàng đợi rỗng, in thông báo và không xóa phần tử.
- Lấy phần tử từ vị trí front.
- Nếu hàng đợi chỉ có một phần tử, đặt lại front và rear về -1.
- Tăng giá trị front để xóa phần tử.

```
void dequeue() {
    if (isEmpty()) {
        cout << "Hang doi dang trong " << endl;
        return;
    }
    int element = queue[front];
    if (front == rear) {
        front = -1;
        rear = -1;
    } else {
        front++;
    }
    cout << "Da xoa phan tu : " << element << endl;
}
```

Hình 2.3. Mô tả xóa phần tử khỏi hàng đợi

2.1.4. Kiểm tra hàng đợi rỗng (isEmpty)

- Kiểm tra xem front có bằng -1 không, nếu có, hàng đợi rỗng.

```
bool isEmpty() {
    return front == -1;
}
```

Hình 2.4. Mô tả hàng đợi rỗng

2.1.5. Kiểm tra hàng đợi đầy (isFull)

- Kiểm tra xem rear có đạt đến maxSize - 1 không, nếu có, hàng đợi đầy.

```
bool isFull() {
    return rear == maxSize - 1;
}
```

Hình 2.5. Mô tả hàng đợi đầy

2.2. Cài đặt hàng đợi bằng danh sách liên kết (Khởi tạo, thêm, xóa, kiểm tra rỗng, đầy)

2.2.1. Khởi tạo hàng đợi

- Constructor khởi tạo front và rear là nullptr.
- Destructor giải phóng bộ nhớ đã cấp phát cho các nút trong danh sách liên kết.

```
Queue() {
    front = nullptr;
    rear = nullptr;
}

~Queue() {
    while (!isEmpty()) {
        dequeue();
    }
}
```

Hình 2.5. Khởi tạo hàng đợi bằng danh sách liên kết

2.2.2. Thêm phần tử vào hàng đợi (enqueue)

- Tạo một nút mới newNode chứa dữ liệu element.
- Nếu hàng đợi rỗng, đặt front và rear đều trỏ đến newNode.
- Nếu không rỗng, thêm newNode vào cuối hàng đợi và cập nhật rear.

```
10  void enqueue(int element) {
11      Node* newNode = new Node();
12      newNode->data = element;
13      newNode->next = nullptr;
14
15      if (isEmpty()) {
16          front = newNode;
17          rear = newNode;
18      } else {
19          rear->next = newNode;
20          rear = newNode;
21      }
22      cout << "Enqueued: " << element << endl;
23  }
```

Hình 2.6. Mô tả thêm phần tử vào hàng đợi bằng danh sách liên kết

2.2.3. Xóa phần tử khỏi hàng đợi (dequeue)

- Nếu hàng đợi rỗng, in thông báo và không xóa phần tử.
- Lấy phần tử từ vị trí front, cập nhật front trỏ đến nút kế tiếp.
- Nếu hàng đợi chỉ còn một phần tử, đặt lại rear về nullptr.
- Giải phóng bộ nhớ của nút vừa bị xóa.

```
1   void dequeue() {
2       if (isEmpty()) {
3           cout << "Queue is empty" << endl;
4           return;
5       }
6
7       Node* temp = front;
8       front = front->next;
9
10      if (front == nullptr) {
11          rear = nullptr;
12      }
13
14      cout << "Dequeued: " << temp->data << endl;
15      delete temp;
16  }
```

Hình 2.7. Mô tả xóa phần tử khỏi hàng đợi bằng danh sách liên kết

2.2.4. Kiểm tra hàng đợi rỗng

- Nếu hàng đợi rỗng, in thông báo và không xóa phần tử.
- Lấy phần tử từ vị trí front, cập nhật front trỏ đến nút kế tiếp.
- Nếu hàng đợi chỉ còn một phần tử, đặt lại rear về nullptr.
- Giải phóng bộ nhớ của nút vừa bị xóa.

```
1   bool isEmpty() {
2       return front == nullptr;
3   }
4
```

Hình 2.8. Mô tả hằng đợi rỗng danh sách liên kết

2.2.5. Kiểm tra hàng đợi đầy

- isEmpty kiểm tra xem front có bằng nullptr không, nếu có, hàng đợi rỗng.

```
6  bool isFull() {
7      return false; //
8  }
```

Hình 2.9. Mô tả hằng đợi đầy danh sách liên kết

2.3. Ứng dụng hàng đợi trong c++

2.3.1. Hàm main

Hàm main trong ngôn ngữ lập trình C++ là điểm khởi đầu của bất kỳ chương trình nào. Khi chương trình được thực thi, hàm main là nơi chương trình bắt đầu chạy.

Hàm main cho chương trình hàng đợi:

```
int main() {
    int size;
    cout << "Nhap so luong phan tu hang doi : ";
    cin >> size;
    Queue queue(size);

    int choice, element;
    while (true) {
        cout << "\n1. Them phan tu \n2. Xoa phan tu \n3. Hien thi cac phan tu \n4. Thoat chuong trinh\n";
        cout << "Vui long chon: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Nhap phan tu vao hang doi : ";
            cin >> element;
            queue.enqueue(element);
            break;
        case 2:
            queue.dequeue();
            break;
        case 3:
            queue.display();
            break;
        case 4:
            return 0;
        default:
            cout << "Lua chon khong hop le, vui long thu lai." << endl;
        }
    }
}
```

Hình 2.10. Mô tả hàm main trong C++

Ngoài ra chúng ta cần thêm hàm hiển thị các phần tử hiện có trong hàng đợi, như sau:

```
    void display() {
        if (isEmpty()) {
            cout << "Hang doi dang trong " << endl;
        } else {
            cout << "Cac phan tu hang doi la : ";
            for (int i = front; i <= rear; i++) {
                cout << queue[i] << " ";
            }
            cout << endl;
        }
    }
};
```

Hình 2.11. Mô tả hàm hiển thị các phần tử hàng đợi trong c++

CHƯƠNG III: ỨNG DỤNG VÀ ĐÁNH GIÁ HIỆU QUẢ CỦA HÀNG ĐỢI

Tóm tắt chương 3

Hàng đợi (Queue) được sử dụng rộng rãi trong nhiều ứng dụng thực tế như quản lý tác vụ trong hệ điều hành, xử lý yêu cầu trong các hệ thống mạng, và điều phối tài nguyên trong các hệ thống đa nhiệm. Đặc biệt, hàng đợi ưu tiên là một biến thể quan trọng, cho phép xử lý các phần tử dựa trên mức độ ưu tiên. Khi đánh giá hiệu quả của hàng đợi, cần xem xét các yếu tố như độ phức tạp thời gian của các thao tác (enqueue, dequeue, peek) và quản lý bộ nhớ. Hàng đợi sử dụng mảng tĩnh có thể gặp hạn chế về bộ nhớ và độ phức tạp thời gian, trong khi hàng đợi sử dụng danh sách liên kết có khả năng mở rộng linh hoạt hơn nhưng có thể tốn thêm chi phí cho việc quản lý bộ nhớ động. Việc lựa chọn cấu trúc hàng đợi phụ thuộc vào yêu cầu cụ thể của ứng dụng về hiệu suất và tài nguyên.

3.1. Các bài toán ứng dụng hàng đợi

3.1.1. Ứng dụng cấu trúc hàng đợi để nhập dãy các số nguyên từ bàn phím và đưa vào hàng đợi, nếu gặp số -1 thì in ra màn hình danh sách các phần tử lấy ra khỏi hàng đợi.

Để nếu gặp -1 thì in ra màn hình danh sách các phần tử lấy ra khỏi hàng đợi, chúng ta thêm:

- displayDequeuedElements: Hàm này duyệt qua tất cả các phần tử trong hàng đợi và in ra các phần tử đó bằng cách gọi dequeue.

- Trong main: Khi nhập phần tử, nếu người dùng nhập -1, gọi phương thức displayDequeuedElements để in ra các phần tử đã xóa khỏi hàng đợi và sau đó quay lại vòng lặp chính để tiếp tục nhận các lệnh khác.

```
84      void displayDequeuedElements() {
85          if (isEmpty()) {
86              cout << "Khong co phan tu nao de xoa " << endl;
87              return;
88          }
89          cout << "Cac phan tu da xoa khoi hang doi la : ";
90          while (!isEmpty()) {
91              cout << dequeue() << " ";
92          }
93          cout << endl;
94      }
95  };
```

Hình 3.1. Mô tả hàm gặp -1 thì in ra màn hình danh sách hàng đợi

3.1.2. Viết hàm tìm phần tử lớn nhất lấy ra khỏi hàng đợi

Để viết hàm tìm phần tử lớn nhất trong hàng đợi và lấy ra khỏi hàng đợi:

- Duyệt qua tất cả các phần tử trong hàng đợi để tìm phần tử lớn nhất.
- Khi duyệt qua, so sánh mỗi phần tử với phần tử lớn nhất hiện tại. Nếu phần tử hiện tại lớn hơn phần tử lớn nhất, cập nhật phần tử lớn nhất thành phần tử hiện tại.
- Khi kết thúc duyệt, phần tử lớn nhất sẽ được tìm thấy.
- Tiến hành loại bỏ phần tử lớn nhất khỏi hàng đợi.

```
96      int findAndDequeueMaxElement() {
97          if (isEmpty()) {
98              cout << "Hang doi dang trong" << endl;
99              return -1;
100         }
101
102         int maxIndex = front;
103         for (int i = front + 1; i <= rear; i++) {
104             if (queue[i] > queue[maxIndex]) {
105                 maxIndex = i;
106             }
107         }
108
109         int maxElement = queue[maxIndex];
110
111         for (int i = maxIndex; i < rear; i++) {
112             queue[i] = queue[i + 1];
113         }
114         rear--;
115
116         if (rear < front) {
117             front = -1;
118             rear = -1;
119         }
120
121         return maxElement;
122     }
123 };
```

Hình 3.2. Mô tả hàm tìm giá trị lớn nhất trong hàng đợi bằng c++

3.1.3. Viết hàm tính tổng các phần tử lấy ra khỏi hàng đợi

Để thêm chức năng tính tổng các phần tử đã được lấy ra khỏi hàng đợi, chúng ta cần:

- Thêm một biến thành viên để lưu trữ tổng các phần tử đã lấy ra.
- Cập nhật phương thức dequeue() để thêm giá trị của phần tử bị lấy ra vào tổng.
- Thêm một phương thức mới để trả về tổng các phần tử đã lấy ra.
- Cập nhật chương trình chính để thêm tùy chọn hiển thị tổng các phần tử đã lấy ra.

```
19          if (rear < front) {
20              front = -1;
21              rear = -1;
22          }
23
24          sumDequeued += maxElement;
25          return maxElement;
26      }
27
28      int getSumDequeued() {
29          return sumDequeued;
30      }
31  };
```

Hình 3.3. Mô tả hàm tính tổng các phần tử ra khỏi hàng đợi

3.1.4. Viết hàm tìm phần tử có giá trị x lấy ra khỏi hàng đợi

Để thêm một hàm tìm và xóa phần tử có giá trị x trong hàng đợi, chúng ta sẽ thực hiện các bước sau:

- Viết một hàm mới findAndDequeueElement(int x) trong lớp Queue.
- Trong hàm này, duyệt qua các phần tử trong hàng đợi để tìm phần tử có giá trị bằng x.
- Nếu tìm thấy phần tử đó, xóa nó khỏi hàng đợi và trả về giá trị của phần tử bị xóa.
- Nếu không tìm thấy, trả về một giá trị chỉ báo (ví dụ: -1).

```
127     int findAndDequeueElement(int x) {
128         if (isEmpty()) {
129             cout << "Hang doi dang trong" << endl;
130             return -1;
131         }
132 
133         int foundIndex = -1;
134         for (int i = front; i <= rear; i++) {
135             if (queue[i] == x) {
136                 foundIndex = i;
137                 break;
138             }
139         }
140 
141         if (foundIndex == -1) {
142             cout << "Khong tim thay phan tu co gia tri " << x << endl;
143             return -1;
144         }
145 
146         int element = queue[foundIndex];
147         for (int i = foundIndex; i < rear; i++) {
148             queue[i] = queue[i + 1];
149         }
150         rear--;
151 
152         if (rear < front) {
153             front = -1;
154             rear = -1;
155         }
156 
157         sumDequeued += element;
158         return element;
159     }
160 
161     int getSumDequeued() {
162         return sumDequeued;
163     }
164 };
```

Hình 3.4. Mô tả hàm lấy giá trị x ra khỏi hàng đợi

3.1.5. Cho 1 chuỗi mô tả các thao tác trên queue gồm các ký tự chữ cái và dấu *

Để thêm hàm xử lý chuỗi ký tự, ta thực hiện như sau:

- Khởi tạo đối tượng queue của lớp Queue.
- Khai báo chuỗi để lưu trữ các thao tác.
- Yêu cầu người dùng nhập chuỗi thao tác.
- Nhập chuỗi thao tác từ người dùng.
- Gọi hàm processOperations để xử lý các thao tác trên hàng đợi.
- Gọi hàm displayQueueContents để hiển thị và loại bỏ tất cả các phần tử còn lại trong hàng đợi.

```cpp
class Queue {
private:
    queue<char> q;

public:
    void enqueue(char c) {
        q.push(c);
        cout << "Da them phan tu: " << c << endl;
    }

    void dequeue() {
        if (q.empty()) {
            cout << "Hang doi dang trong" << endl;
        } else {
            char c = q.front();
            q.pop();
            cout << "Da lay phan tu: " << c << endl;
        }
    }

    void processOperations(const string& operations) {
        for (char op : operations) {
            if (op == '*') {
                dequeue();
            } else {
                enqueue(op);
            }
        }
    }

    void displayQueueContents() {
        cout << "Noi dung con lai trong hang doi: ";
        while (!q.empty()) {
            cout << q.front();
            q.pop();
        }
        cout << endl;
    }
};
int main() {
    Queue queue;
    string operations;

    cout << "Nhap chuoi thao tac tren queue: ";
    cin >> operations;

    queue.processOperations(operations);
    queue.displayQueueContents();

    return 0;
}
```

*Hình 3.5. Mô tả thao tác trên queue gồm các ký tự chữ cái và dấu **

3.2. Cài đặt danh sách sinh viên sử dụng hàng đợi (Nâng cao)

Để cài đặt danh sách sinh viên sử dụng hàng đợi, ta cần thiết kế một lớp Sinhvien để lưu trữ thông tin của sinh viên, và một lớp Queue để quản lý danh sách sinh viên dưới dạng hàng đợi:

- Lớp Sinhvien: Lớp này lưu trữ thông tin cơ bản của một sinh viên bao gồm Mã sv, tên, ngày sinh và giới tính, tuổi.

```cpp
class Sinhvien {
public:
    string masv;
    string ten;
    string ngaysinh;
    string gioitinh;
    int tuoi;

    Sinhvien() : masv(""), ten(""), ngaysinh(""), gioitinh(""), tuoi(0) {}

    Sinhvien(string masv, string ten, string ngaysinh, string gioitinh, int tuoi) : masv(masv), ten(ten), ngaysinh(ngaysinh), gioitinh(gioitinh), tuoi(tuoi

    void hienthi() {
        cout << "MaSV: " << masv << ", TenSV: " << ten << ", Ngay sinh: " << ngaysinh << ", Gioi tinh: " << gioitinh << ", Tuoi: " << tuoi << endl;
```

Hình 3.6. Mô tả khởi tạo lưu trữ thông tin cơ bản sinh viên

- Lớp Queue: Lớp này quản lý một hàng đợi các đối tượng Sinhvien. Nó có các phương thức để thêm (them), xóa (xoa) và hiển thị (hienthi) các sinh viên.

```
22  class Queue {
23  private:
24      int front;
25      int rear;
26      int maxSize;
27      Sinhvien* queue;
28
29  public:
30      Queue(int size) {
31          maxSize = size;
32          queue = new Sinhvien[maxSize];
33          front = -1;
34          rear = -1;
35      }
36
37      ~Queue() {
38          delete[] queue;
39      }
```

Hình 3.7. Mô tả lớp queue quản lý hàng đợi các đối tượng sinh viên

- Khởi tạo kiểm tra rỗng, đầy của danh sách sinh viên.

```
41      bool rong() {
42          return front == -1;
43      }
44
45      bool day() {
46          return rear == maxSize - 1;
47      }
```

Hình 3.8. Mô tả kiểm tra rỗng, đầy trong danh sách sinh viên

- Hàm thêm trong danh sách sinh viên.

```
49      void them(Sinhvien sinhvien) {
50          if (day()) {
51              cout << "Danh sach day !!!" << endl;
52              return;
53          }
54          if (rong()) {
55              front = 0;
56          }
57          rear++;
58          queue[rear] = sinhvien;
59          cout << "Da them sinh vien: " << sinhvien.ten << endl;
60      }
```

Hình 3.9. Mô tả hàm thêm trong danh sách sinh viên

- Hàm xóa trong danh sách sinh viên.

```
62      Sinhvien xoa() {
63          if (rong()) {
64              cout << "Danh sach dang trong" << endl;
65              return Sinhvien();
66          }
67          Sinhvien sinhvien = queue[front];
68          if (front == rear) {
69              front = -1;
70              rear = -1;
71          } else {
72              front++;
73          }
74          return sinhvien;
75      }
```

Hình 3.10. Mô tả hàm xóa trong danh sách sinh viên

- Hàm hiển thị để in danh sách sách sinh viên ra màn hình.

```
77  void hienthi() {
78      if (rong()) {
79          cout << "Danh sach dang trong" << endl;
80      } else {
81          cout << "Danh sach sinh vien trong hang doi la:" << endl;
82          for (int i = front; i <= rear; i++) {
83              queue[i].hienthi();
84          }
85      }
86  }
```

Hình 3.11. Mô tả hàm hiển thị danh sách ra màn hình

- Hàm để tìm và xóa sinh viên theo mã sinh viên trong danh sách sinh viên.

```
88  Sinhvien timVaXoaTheoMaSV(string masv) {
89      if (rong()) {
90          cout << "Danh sach dang trong" << endl;
91          return Sinhvien();
92      }
93
94      for (int i = front; i <= rear; i++) {
95          if (queue[i].masv == masv) {
96              Sinhvien sinhvien = queue[i];
97
98              // Xoa sinh vien tai vi tri i
99              for (int j = i; j < rear; j++) {
100                 queue[j] = queue[j + 1];
101             }
102             rear--;
103
104             if (rear < front) {
105                 front = -1;
106                 rear = -1;
107             }
108
109             return sinhvien;
110         }
111     }
112
113     cout << "Khong tim thay sinh vien co ma: " << masv << endl;
114     return Sinhvien();
115 }
```

Hình 3.12. Mô tả tìm và xóa sinh viên theo mã sinh viên

- Hàm để tìm và xóa sinh viên theo tên sinh viên trong danh sách sinh viên.

```
117     Sinhvien timVaXoaTheoTen(string ten) {
118         if (rong()) {
119             cout << "Danh sach dang trong" << endl;
120             return Sinhvien();
121         }
122
123         for (int i = front; i <= rear; i++) {
124             if (queue[i].ten == ten) {
125                 Sinhvien sinhvien = queue[i];
126
127                 // Xoa sinh vien tai vi tri i
128                 for (int j = i; j < rear; j++) {
129                     queue[j] = queue[j + 1];
130                 }
131                 rear--;
132
133                 if (rear < front) {
134                     front = -1;
135                     rear = -1;
136                 }
137
138                 return sinhvien;
139             }
140         }
141
142         cout << "Khong tim thay sinh vien co ten: " << ten << endl;
143         return Sinhvien();
144     }
145 };
```

Hình 3.13. Mô tả tìm và xóa sinh viên theo tên sinh viên

- Chương trình chính (Hàm main) : Chương trình này cung cấp một menu để người dùng có thể thực hiện các thao tác thêm sinh viên vào hàng đợi, xóa sinh viên khỏi hàng đợi, và hiển thị danh sách sinh viên hiện tại trong hàng đợi.

```
147 int main() {
148     int size;
149     cout << "Nhap so luong sinh vien toi da: ";
150     cin >> size;
151     Queue queue(size);
152 
153     int chon;
154     while (true) {
155         cout << "\n1. Them sinh vien\n2. Xoa sinh vien\n3. Hien thi danh sach sinh
156         cout << "Vui long chon: ";
157         cin >> chon;
158 
159         switch (chon) {
160             case 1: {
161                 string masv, ten, ngaysinh, gioitinh;
162                 int tuoi;
163                 cout << "Nhap ma sinh vien: ";
164                 cin >> masv;
165                 cout << "Nhap ten sinh vien: ";
166                 cin.ignore();
167                 getline(cin, ten);
168                 cout << "Nhap ngay sinh: ";
169                 getline(cin, ngaysinh);
170                 cout << "Nhap gioi tinh: ";
171                 getline(cin, gioitinh);
172                 cout << "Nhap tuoi sinh vien: ";
173                 cin >> tuoi;
174                 Sinhvien sinhvien(masv, ten, ngaysinh, gioitinh, tuoi);
175                 queue.them(sinhvien);
176                 break;
177             }
178             case 2: {
179                 Sinhvien sinhvien = queue.xoa();
180                 if (sinhvien.masv != "") {
181                     cout << "Da xoa sinh vien: " << sinhvien.ten << endl;
182                 }
183                 break;
184             }
185             case 3:
186                 queue.hienthi();
187                 break;
188             case 4: {
189                 string masv;
190                 cout << "Nhap ma sinh vien can xoa: ";
191                 cin >> masv;
192                 Sinhvien sinhvien = queue.timVaXoaTheoMaSV(masv);
193                 if (sinhvien.masv != "") {
194                     cout << "Sinh vien co ma " << masv << " da duoc xoa." << endl;
195                 }
196                 break;
197             }
198             case 5: {
199                 string ten;
200                 cout << "Nhap ten sinh vien can xoa: ";
201                 cin.ignore();
202                 getline(cin, ten);
203                 Sinhvien sinhvien = queue.timVaXoaTheoTen(ten);
204                 if (sinhvien.masv != "") {
205                     cout << "Sinh vien co ten " << ten << " da duoc xoa." << endl;
206                 }
207                 break;
208             }
209             case 6:
210                 return 0;
211             default:
212                 cout << "Lua chon khong hop le, vui long thu lai." << endl;
213         }
214     }
215 }
```

Hình 3.14. Mô tả hàm main trong cấu trúc danh sách sinh viên sử dụng hàng đợi

3.3. Đánh giá và hướng phát triển trong tương lai

3.3.1. Đánh giá

Ưu điểm:

- Đơn giản và hiệu quả: Hàng đợi cung cấp cơ chế đơn giản và hiệu quả cho việc quản lý dữ liệu theo thứ tự vào trước ra trước (FIFO).

- Ứng dụng đa dạng: Được sử dụng rộng rãi trong các hệ thống mạng, hệ điều hành, và nhiều ứng dụng khác để quản lý tác vụ, điều phối tài nguyên, và xử lý dữ liệu theo dòng chảy.

Hạn chế:

- Giới hạn bộ nhớ: Hàng đợi tĩnh có thể gặp hạn chế về bộ nhớ và cần phải định trước kích thước, dẫn đến lãng phí hoặc thiếu hụt bộ nhớ.

- Hiệu suất: Hàng đợi động sử dụng danh sách liên kết có thể tốn chi phí quản lý bộ nhớ động, dẫn đến giảm hiệu suất trong một số trường hợp.

3.3.2. Hướng phát triển trong tương lai

- Cải thiện cấu trúc dữ liệu: Phát triển các cấu trúc dữ liệu mới như hàng đợi ưu tiên với độ phức tạp thấp hơn, giúp tăng hiệu suất xử lý trong các ứng dụng đòi hỏi tính ưu tiên cao.

- Ứng dụng trong lĩnh vực mới: Khám phá và tích hợp hàng đợi vào các lĩnh vực mới như học máy, trí tuệ nhân tạo và xử lý dữ liệu lớn, nơi yêu cầu quản lý dữ liệu theo thứ tự và độ ưu tiên cao.

- Tối ưu hóa bộ nhớ: Nghiên cứu và phát triển các kỹ thuật quản lý bộ nhớ tiên tiến, giúp hàng đợi trở nên linh hoạt và hiệu quả hơn trong việc sử dụng bộ nhớ động.

- Tích hợp với công nghệ hiện đại: Kết hợp hàng đợi với các công nghệ hiện đại như blockchain, IoT và các hệ thống phân tán để cải thiện khả năng quản lý dữ liệu và tác vụ trong môi trường phân tán và đa người dùng.

KẾT LUẬN

Hàng đợi là một cấu trúc dữ liệu cơ bản nhưng mạnh mẽ, có vai trò quan trọng trong nhiều lĩnh vực của khoa học máy tính và ứng dụng thực tế. Với nguyên tắc hoạt động vào trước ra trước (FIFO), hàng đợi cung cấp giải pháp hiệu quả cho việc quản lý dữ liệu và tác vụ theo thứ tự thời gian, đảm bảo tính nhất quán và thứ tự xử lý.

1. Các kết quả đạt được

- Quản lý dữ liệu hiệu quả: Hàng đợi giúp tổ chức và quản lý dữ liệu theo nguyên tắc FIFO, đảm bảo tính nhất quán và đúng thứ tự xử lý.

- Tăng hiệu suất xử lý: Hàng đợi giúp tối ưu hóa thời gian xử lý cho các tác vụ theo thứ tự đến, giảm thiểu thời gian chờ đợi và tăng hiệu suất hệ thống.

- Ứng dụng rộng rãi: Hàng đợi có ứng dụng trong nhiều lĩnh vực, từ hệ thống điều khiển, mạng máy tính, đến các ứng dụng hàng ngày như hàng chờ mua vé, xếp hàng tại các dịch vụ công cộng.

- Giải quyết vấn đề: Hàng đợi cung cấp một cơ chế linh hoạt để giải quyết nhiều vấn đề trong lập trình và thiết kế hệ thống, như quản lý tài nguyên, xử lý sự kiện, hoặc xử lý đồng thời.

- Tích hợp dễ dàng: Hàng đợi là một cấu trúc dữ liệu đơn giản và linh hoạt, dễ dàng tích hợp và triển khai trong nhiều ngôn ngữ lập trình và môi trường phát triển khác nhau.

2. Ưu và nhược điểm của đề tài

Ưu điểm

- Ứng dụng rộng rãi: Hàng đợi được sử dụng trong nhiều lĩnh vực khác nhau, từ lập trình máy tính đến các hệ thống thời gian thực và các ứng dụng hàng ngày như xếp hàng tại các điểm dịch vụ.

- Giải quyết các vấn đề phức tạp:Hàng đợi

- Tối ưu hóa hiệu suất: Hàng đợi giúp tối ưu hóa thời gian xử lý và giảm thiểu thời gian chờ đợi, từ đó tăng hiệu suất và cải thiện trải nghiệm người dùng.

Nhược điểm

- Khả năng hiểu quả phụ thuộc vào triển khai: Hiệu suất của hàng đợi có thể bị ảnh hưởng bởi cách triển khai và thao tác trên hàng đợi, như việc sử dụng một cách không hiệu quả hoặc không tuân thủ nguyên tắc FIFO.

- Độ phức tạp của thuật toán: Một số thuật toán liên quan đến hàng đợi có độ phức tạp cao, đặc biệt là trong trường hợp sử dụng hàng đợi ưu tiên (priority queue), có thể ảnh hưởng đến hiệu suất của hệ thống.

TÀI LIỆU THAM KHẢO

Tiếng Việt

1. Đỗ Xuân Lôi, 2010, "*Cấu trúc dữ liệu và giải thuật*". Đại học Quốc gia Hà Nội.

Tiếng Anh

2. Thomas H Cormen , Charles E Leiserson , Ronald L Rivest , Clifford Stein, 2001, "*Introduction To Algorithms*". Mit

Các website

1. https://viettuts.vn/cau-truc-du-lieu-va-giai-thuat/cau-truc-du-lieu-hang-doi-

2.https://vietjack.com/cau-truc-du-lieu-va-giai-thuat/cau-truc-du-lieu-hang-doi.jsp#google_vignette

PHỤ LỤC

1. Code chạy chương trình hàng đợi (Queue)

```cpp
#include <iostream>
using namespace std;
class Queue {
private:
    int front;
    int rear;
    int maxSize;
    int* queue;
    int sumDequeued;
public:
    Queue(int size) {
        maxSize = size;
        queue = new int[maxSize];
        front = -1;
        rear = -1;
        sumDequeued = 0;
    }
    ~Queue() {
        delete[] queue;
    }
    bool isEmpty() {
        return front == -1;
    }
    bool isFull() {
        return rear == maxSize - 1;
    }
    void enqueue(int element) {
        if (isFull()) {
            cout << "Hang doi day !!!" << endl;
            cout << "1. Xoa phan tu dau tien va them moi" << endl;
            cout << "2. Bo qua phan tu moi" << endl;
            int option;
            cin >> option;
            switch (option) {
                case 1:
                    dequeue();
                    break;
                case 2:
                    return;
                default:
                    cout << "Lua chon khong hop le, bo qua phan tu moi." << endl;
                    return;
            }
        }
        if (isEmpty()) {
            front = 0;
        }
        rear++;
        queue[rear] = element;
        cout << "Da them phan tu : " << element << endl;
    }
    int dequeue() {
        if (isEmpty()) {
            cout << "Hang doi dang trong " << endl;
            return -1;
        }
        int element = queue[front];
        sumDequeued += element;
        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front++;
        }
        return element;
    }
    void display() {
        if (isEmpty()) {
            cout << "Hang doi dang trong " << endl;
        } else {
            cout << "Cac phan tu hang doi la : ";
            for (int i = front; i <= rear; i++) {
                cout << queue[i] << " ";
            }
            cout << endl;
        }
    }
    void displayDequeuedElements() {
        if (isEmpty()) {
            cout << "Khong co phan tu nao de xoa " << endl;
            return;
        }
```

```
83          cout << "Cac phan tu da xoa khoi hang doi la : ";
84          while (!isEmpty()) {
85              cout << dequeue() << " ";
86          }
87          cout << endl;
88      }
89      int findAndDequeueMaxElement() {
90          if (isEmpty()) {
91              cout << "Hang doi dang trong" << endl;
92              return -1;
93          }
94          int maxIndex = front;
95          for (int i = front + 1; i <= rear; i++) {
96              if (queue[i] > queue[maxIndex]) {
97                  maxIndex = i;
98              }
99          }
100         int maxElement = queue[maxIndex];
101
102         for (int i = maxIndex; i < rear; i++) {
103             queue[i] = queue[i + 1];
104         }
105         rear--;
106
107         if (rear < front) {
108             front = -1;
109             rear = -1;
110         }
111         sumDequeued += maxElement;
112         return maxElement;
113     }
114     int findAndDequeueElement(int x) {
115         if (isEmpty()) {
116             cout << "Hang doi dang trong" << endl;
117             return -1;
118         }
119         int foundIndex = -1;
120         for (int i = front; i <= rear; i++) {
121             if (queue[i] == x) {
122                 foundIndex = i;
123                 break;
124             }
125         }
126         if (foundIndex == -1) {
127             cout << "Khong tim thay phan tu co gia tri " << x << endl;
128             return -1;
129         }
130         int element = queue[foundIndex];
131         for (int i = foundIndex; i < rear; i++) {
132             queue[i] = queue[i + 1];
133         }
134         rear--;
135         if (rear < front) {
136             front = -1;
137             rear = -1;
138         }
139         sumDequeued += element;
140         return element;
141     }
142     int findAndDequeueMinElement() {
143         if (isEmpty()) {
144             cout << "Hang doi dang trong" << endl;
145             return -1;
146         }
147         int minIndex = front;
148         for (int i = front + 1; i <= rear; i++) {
149             if (queue[i] < queue[minIndex]) {
150                 minIndex = i;
151             }
152         }
153         int minElement = queue[minIndex];
154
155         for (int i = minIndex; i < rear; i++) {
156             queue[i] = queue[i + 1];
157         }
158         rear--;
159         if (rear < front) {
160             front = -1;
161             rear = -1;
162         }
163         sumDequeued += minElement;
164         return minElement;
```

```
    }
    int getSumDequeued() {
        return sumDequeued;
    }
};

int main() {
    int size;
    cout << "Nhap so luong phan tu hang doi : ";
    cin >> size;
    Queue queue(size);
    int choice, element;
    while (true) {
        cout << "\n1. Them phan tu\n2. Xoa phan tu\n3. Hien thi cac phan tu\n4. Tim va xoa phan tu lon nhat\n5
        cout << "Vui long chon: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Nhap cac so nguyen vao hang doi (nhap -1 de xoa va hien thi cac phan tu da xoa): ";
                cin >> element;
                if (element == -1) {
                    queue.displayDequeuedElements();
                } else {
                    queue.enqueue(element);
                }
                break;
            case 2:
                queue.dequeue();
                break;
            case 3:
                queue.display();
                break;
            case 4: {
                int maxElement = queue.findAndDequeueMaxElement();
                if (maxElement != -1) {
                    cout << "Phan tu lon nhat da lay ra khoi hang doi la: " << maxElement << endl;
                }
                break;
            }
             case 5: {
                int minElement = queue.findAndDequeueMinElement();
                if (minElement != -1) {
                    cout << "Phan tu nho nhat da lay ra khoi hang doi la: " << minElement << endl;
                }
                break;
            }
             case 6:
                cout << "Nhap gia tri cua phan tu can xoa: ";
                cin >> element;
                if (queue.findAndDequeueElement(element) != -1) {
                    cout << "Da xoa phan tu co gia tri " << element << " khoi hang doi." << endl;
                }
                break;
            case 7:
                cout << "Nhap cac so nguyen vao hang doi (nhap -1 de xoa va hien thi cac phan tu da xoa): "
                while (true) {
                    cin >> element;
                    if (element == -1) {
                        queue.displayDequeuedElements();
                        break;
                    } else {
                        queue.enqueue(element);
                    }
                }
                break;
            case 8:
                cout << "Tong cac phan tu da lay ra khoi hang doi la: " << queue.getSumDequeued() << endl;
                break;
            case 9:
                return 0;
            default:
                cout << "Lua chon khong hop le, vui long thu lai." << endl;
        }
    }
}
```

2. Code chạy chương trình cài đặt cấu trúc sinh viên gồm Họ tên, Ngày sinh, Giới tính, lần lượt đưa sinh viên vào hàng đợi.

```
#include <iostream>
#include <string>
using namespace std;
class Sinhvien {
public:
    string masv;
    string ten;
    string ngaysinh;
    string gioitinh;
    int tuoi;
    Sinhvien() : masv(""), ten(""), ngaysinh(""), gioitinh(""), tuoi(0) {}
    Sinhvien(string masv, string ten, string ngaysinh, string gioitinh, int tuoi) : masv(masv), ten(ten)
    void hienthi() {
        cout << "MaSV: " << masv << ", TenSV: " << ten << ", Ngay sinh: " << ngaysinh << ", Gioi tinh: "
    }
};

class Queue {
private:
    int front;
    int rear;
    int maxSize;
    Sinhvien* queue;
public:
    Queue(int size) {
        maxSize = size;
        queue = new Sinhvien[maxSize];
        front = -1;
        rear = -1;
    }
    ~Queue() {
        delete[] queue;
    }
    bool rong() {
        return front == -1;
    }
    bool day() {
        return rear == maxSize - 1;
    }
    void them(Sinhvien sinhvien) {
        if (day()) {
            cout << "Danh sach day !!!" << endl;
            return;
        }
        if (rong()) {
            front = 0;
        }
        rear++;
        queue[rear] = sinhvien;
        cout << "Da them sinh vien: " << sinhvien.ten << endl;
    }

    Sinhvien xoa() {
        if (rong()) {
            cout << "Danh sach dang trong" << endl;
            return Sinhvien();
        }
        Sinhvien sinhvien = queue[front];
        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front++;
        }
        return sinhvien;
    }
    void hienthi() {
        if (rong()) {
            cout << "Danh sach dang trong" << endl;
        } else {
            cout << "Danh sach sinh vien trong hang doi la:" << endl;
            for (int i = front; i <= rear; i++) {
                queue[i].hienthi();
            }
        }
    }
    Sinhvien timVaXoaTheoMaSV(string masv) {
        if (rong()) {
            cout << "Danh sach dang trong" << endl;
            return Sinhvien();
        }

        for (int i = front; i <= rear; i++) {
            if (queue[i].masv == masv) {
```

```
            Sinhvien sinhvien = queue[i];

            // Xoa sinh vien tai vi tri i
            for (int j = i; j < rear; j++) {
                queue[j] = queue[j + 1];
            }
            rear--;

            if (rear < front) {
                front = -1;
                rear = -1;
            }

            return sinhvien;
        }
    }
    cout << "Khong tim thay sinh vien co ma: " << masv << endl;
    return Sinhvien();
}
Sinhvien timVaXoaTheoTen(string ten) {
    if (rong()) {
        cout << "Danh sach dang trong" << endl;
        return Sinhvien();
    }
    for (int i = front; i <= rear; i++) {
        if (queue[i].ten == ten) {
            Sinhvien sinhvien = queue[i];
            for (int j = i; j < rear; j++) {
                queue[j] = queue[j + 1];
            }
            rear--;
            if (rear < front) {
                front = -1;
                rear = -1;
            }
            return sinhvien;
        }
    }
    cout << "Khong tim thay sinh vien co ten: " << ten << endl;
    return Sinhvien();
}
};

int main() {
    int size;
    cout << "Nhap so luong sinh vien toi da: ";
    cin >> size;
    Queue queue(size);
    int chon;
    while (true) {
        cout << "\n1. Them sinh vien\n2. Xoa sinh vien\n3. Hien thi danh sach sinh vien\n4. Tim va xoa
        cout << "Vui long chon: ";
        cin >> chon;
        switch (chon) {
            case 1: {
                string masv, ten, ngaysinh, gioitinh;
                int tuoi;
                cout << "Nhap ma sinh vien: ";
                cin >> masv;
                cout << "Nhap ten sinh vien: ";
                cin.ignore();
                getline(cin, ten);
                cout << "Nhap ngay sinh: ";
                getline(cin, ngaysinh);
                cout << "Nhap gioi tinh: ";
                getline(cin, gioitinh);
                cout << "Nhap tuoi sinh vien: ";
                cin >> tuoi;
                Sinhvien sinhvien(masv, ten, ngaysinh, gioitinh, tuoi);
                queue.them(sinhvien);
                break;
            }
            case 2: {
                Sinhvien sinhvien = queue.xoa();
                if (sinhvien.masv != "") {
                    cout << "Da xoa sinh vien: " << sinhvien.ten << endl;
                }
                break;
            }
            case 3:
                queue.hienthi();
                break;
            case 4: {
                string masv;
```

```
169                 cout << "Nhap ma sinh vien can xoa: ";
170                 cin >> masv;
171                 Sinhvien sinhvien = queue.timVaXoaTheoMaSV(masv);
172                 if (sinhvien.masv != "") {
173                     cout << "Sinh vien co ma " << masv << " da duoc xoa." << endl;
174                 }
175                 break;
176             }
177             case 5: {
178                 string ten;
179                 cout << "Nhap ten sinh vien can xoa: ";
180                 cin.ignore();
181                 getline(cin, ten);
182                 Sinhvien sinhvien = queue.timVaXoaTheoTen(ten);
183                 if (sinhvien.masv != "") {
184                     cout << "Sinh vien co ten " << ten << " da duoc xoa." << endl;
185                 }
186                 break;
187             }
188             case 6:
189                 return 0;
190             default:
191                 cout << "Lua chon khong hop le, vui long thu lai." << endl;
192         }
193     }
194 }
```